

计算方法（A）大作业

姓名：

班级：

专业：

学号：

共轭梯度法

一、 算法原理

共轭梯度法是把求解线性方程组的问题转化为求解一个与之等价的二次函数极小化的问题，因此从任意给定的初始点出发，沿一组关于矩阵 A 的共轭方向进行线性搜索，在无舍入无差的假定下，最多迭代 n (其中 n 为矩阵 A 的阶数) 次就可求得二次函数的极小点，也就求得了线性方程组 $Ax=b$ 的解。

下述定理给出了求系数矩阵 A 是对称正定矩阵的线性方程组 $Ax=b$ 的解与求二次函数 $f(x) = \frac{1}{2}x^T Ax - b^T x$ 极小点的等价性。

定理 3.4.1 设 A 是 n 阶对称正定矩阵，则 x^* 是方程组 $Ax=b$ 的解的充分必要条件是 x^* 是二次函数 $f(x) = \frac{1}{2}x^T Ax - b^T x$ 的极小点，即

$$Ax^* = b \Leftrightarrow f(x^*) = \min_{x \in R^n} f(x)$$

证明：充分性. 设 x^* 是 $f(x)$ 的极小点，则由无约束最优化问题最优解的必要条件知

$$\nabla f(x^*) = Ax^* - b$$

即 x^* 是方程组 $Ax=b$ 的解。

必要性. 若 x^* 是方程组 $Ax=b$ 的解，即 $Ax^*=b$ ，注意到 A 是对称正定矩阵，故 $\forall x \in R^n$ 有

$$\begin{aligned} f(x) - f(x^*) &= \frac{1}{2}x^T Ax - b^T x - \frac{1}{2}x^T Ax^* + b^T x^* \\ &= \frac{1}{2}(x^T Ax - 2b^T x + x^{*T} Ax^*) - x^{*T} Ax^* + b^T x^* \\ &= \frac{1}{2}(x^T Ax - 2(Ax^*)^T x + x^{*T} Ax^*) - (Ax^* - b)^T x^* \\ &= \frac{1}{2}(x - x^*)^T A(x - x^*) \\ &\geq 0 \end{aligned}$$

即 x^* 是 $f(x)$ 的极小点，进而由 A 是正定矩阵知， x^* 是 $f(x)$ 的最小点。证毕。

共轭梯度法在形式上具有迭代法的特征，在给定初始值情况下，根据迭代公式：

$$x^{(k+1)} = x^{(k)} + \alpha_k d^{(k)}$$

产生的迭代序列 $x^{(1)}, x^{(2)}, x^{(3)} \dots$ 在无舍入误差假定下，最多经过 n 次迭代，就可求得 $f(x)$ 的最小值，也就是方程 $Ax=b$ 的解。

共轭梯度法中关键的两点是，确定迭代格式中的搜索向量 $d^{(k)}$ 和最佳步长 α_k ($\alpha_k \geq 0$)。实际上，搜索方向 $d^{(k)}$ 是关于矩阵 A 的共轭向量，在迭代中逐步构造之。步长 α_k 的确定原则是给定迭代点 $x^{(k)}$ 和搜索方向 $d^{(k)}$ 后，要求选取非负实数 α_k ，使得 $f(x^{(k)} + \alpha_k d^{(k)})$ 达到最小，即选择 α_k ，满足

$$f(x^{(k)} + \alpha_k d^{(k)}) = \min_{0 \leq \alpha} f(x^{(k)} + \alpha d^{(k)})$$

下面首先导出最佳步长 α_k 的计算公式。

假设迭代点 $x^{(k)}$ 和搜索方向 $d^{(k)}$ 已经给定， α_k 可以通过一元函数 $\phi(\alpha) = f(x^{(k)} + \alpha d^{(k)})$ 的极小化

$$\min_{0 \leq \alpha} \phi(\alpha) = f(x^{(k)} + \alpha d^{(k)})$$

来求得，根据多元复合函数的求导法则得：

$$\dot{\phi}(\alpha) = \nabla f(x^{(k)} + \alpha d^{(k)})^T d^{(k)}$$

令

$$\begin{aligned} 0 = \dot{\phi}(\alpha) &= \nabla f(x^{(k)} + \alpha d^{(k)})^T d^{(k)} = [A(x^{(k)} + \alpha d^{(k)}) - b]^T d^{(k)} \\ &= (Ax^{(k)} - b + \alpha Ad^{(k)})^T d^{(k)} = (-r^{(k)} + \alpha Ad^{(k)})^T d^{(k)} \end{aligned}$$

其中 $r^{(k)} = b - Ax^{(k)}$ 。由此解得最佳步长

$$\alpha_k = \frac{r^{(k)T} d^{(k)}}{d^{(k)T} A d^{(k)}} \quad (3.4.4)$$

下面确定搜索方向 $d^{(k)}$ 。

给定初始向量 $x^{(0)}$ 后，由于负梯度方向是函数下降最快的方向，故第1次迭代取搜索方向 $d^{(0)} = r^{(0)} = -\nabla f(x^{(0)}) = b - Ax^{(0)}$ 。令

$$x^{(1)} = x^{(0)} + \alpha_0 d^{(0)}$$

其中 $\alpha_0 = \frac{r^{(0)T} d^{(0)}}{d^{(0)T} A d^{(0)}}$ 。第2次迭代时，从 $x^{(1)}$ 出发的搜索方向不再取 $r^{(1)}$ ，而是选取 $d^{(1)} = r^{(1)} + \beta_0 d^{(0)}$ ，使得 $d^{(1)}$ 与 $d^{(0)}$ 是关于矩阵A的共轭向量，即要求 $d^{(1)}$ 满足 $(d^{(1)}, Ad^{(0)}) = 0$ ，由此可求得参数 β_0 ：

$$\beta_0 = -\frac{r^{(1)T} A d^{(0)}}{d^{(0)T} A d^{(0)}}$$

然后从 $x^{(1)}$ 出发，沿 $d^{(1)}$ 进行搜索得到

$$x^{(2)} = x^{(1)} + \alpha_1 d^{(1)}$$

其中 α_1 由式(3.4.4)确定。

一般地，设已经求出 $x^{(k+1)} = x^{(k)} + \alpha_k d^{(k)}$ ，计算 $r^{(k+1)} = b - Ax^{(k+1)}$ 。令 $d^{(k+1)} = r^{(k+1)} + \beta_k d^{(k)}$ ，选取 β_k ，使得 $d^{(k+1)}$ 和 $d^{(k)}$ 是关于A的共轭向量，即要求 $d^{(k+1)}$ 满足 $(d^{(k+1)}, Ad^{(k)}) = 0$ 。由此可得：

$$\beta_k = -\frac{r^{(k+1)T} A d^{(k)}}{d^{(k)T} A d^{(k)}}$$

从而确定了搜索方向 $d^{(k+1)}$ 。

综上所述，共轭梯度法的计算公式为

$$\left\{ \begin{array}{l} d^{(0)} = r^{(0)} = b - Ax^{(0)}, \\ \alpha_k = \frac{r^{(k)T}d^{(k)}}{d^{(k)T}Ad^{(k)}}, \\ x^{(k+1)} = x^{(k)} + \alpha_k d^{(k)}, \\ r^{(k+1)} = b - Ax^{(k+1)}, \\ \beta_k = -\frac{r^{(k+1)T}Ad^{(k)}}{d^{(k)T}Ad^{(k)}}, \\ d^{(k+1)} = r^{(k+1)} + \beta_k d^{(k)}. \end{array} \right.$$

利用定理 3.4.3，可以将 α_k 和 β_k 的表达式简化为

$$\alpha_k = \frac{r^{(k)T}r^{(k)}}{d^{(k)T}Ad^{(k)}} \\ \beta_k = \frac{r^{(k+1)T}r^{(k+1)}}{r^{(k)T}r^{(k)}}$$

定理 3.4.3 设分别是共轭梯度法中产生的非零残向量和搜索方向，则

- (1) $(r^{(k)}, d^{(k-1)}) = 0$
- (2) $(r^{(k)}, d^{(k)}) = (r^{(k)}, r^{(k)})$
- (3) $r^{(0)}, r^{(1)}, \dots, r^{(k)} (k \leq n-1)$ 是正交向量组， $d^{(0)}, d^{(1)}, \dots, d^{(k)} (k \leq n-1)$ 是关于 A 的共轭向量组。

关于共轭向量法的误差有如下定理：

定理 3.4.5 设是用共轭梯度法求得的迭代序列，则有误差估计式

$$\|x^{(k)} - x^*\|_A \leq 2 \left(\frac{\sqrt{K} - 1}{\sqrt{K} + 1} \right)^k \|x^{(0)} - x^*\|_A$$

其中范数 $\|x\|_A = \sqrt{x^T A x}$, $K = \text{Cond}_2(A)$.

若 $r^{(k)} = 0$ ，则 $x^{(k)}$ 就是线性方程组的解，故在计算过程中将 $\|r^{(k+1)}\| \leq \varepsilon$ 作为迭代终止的条件。

共轭梯度法的计算过程如下：

- (1) 给定初始近似向量 $x^{(0)}$ 以及精度 $\varepsilon > 0$;
- (2) 计算 $r^{(0)} = b - Ax^{(0)}$ ，取 $d^{(0)} = r^{(0)}$;
- (3) for k=0 to n-1 do
 - (i) $\alpha_k = \frac{r^{(k)T}d^{(k)}}{d^{(k)T}Ad^{(k)}}$;
 - (ii) $x^{(k+1)} = x^{(k)} + \alpha_k d^{(k)}$;
 - (iii) $r^{(k+1)} = b - Ax^{(k+1)}$;
 - (iv) 若 $\|r^{(k+1)}\| \leq \varepsilon$ 或 $k+1=n$ ，则输出近似解 $x^{(k+1)}$ ，停止；否则，转 (v)；

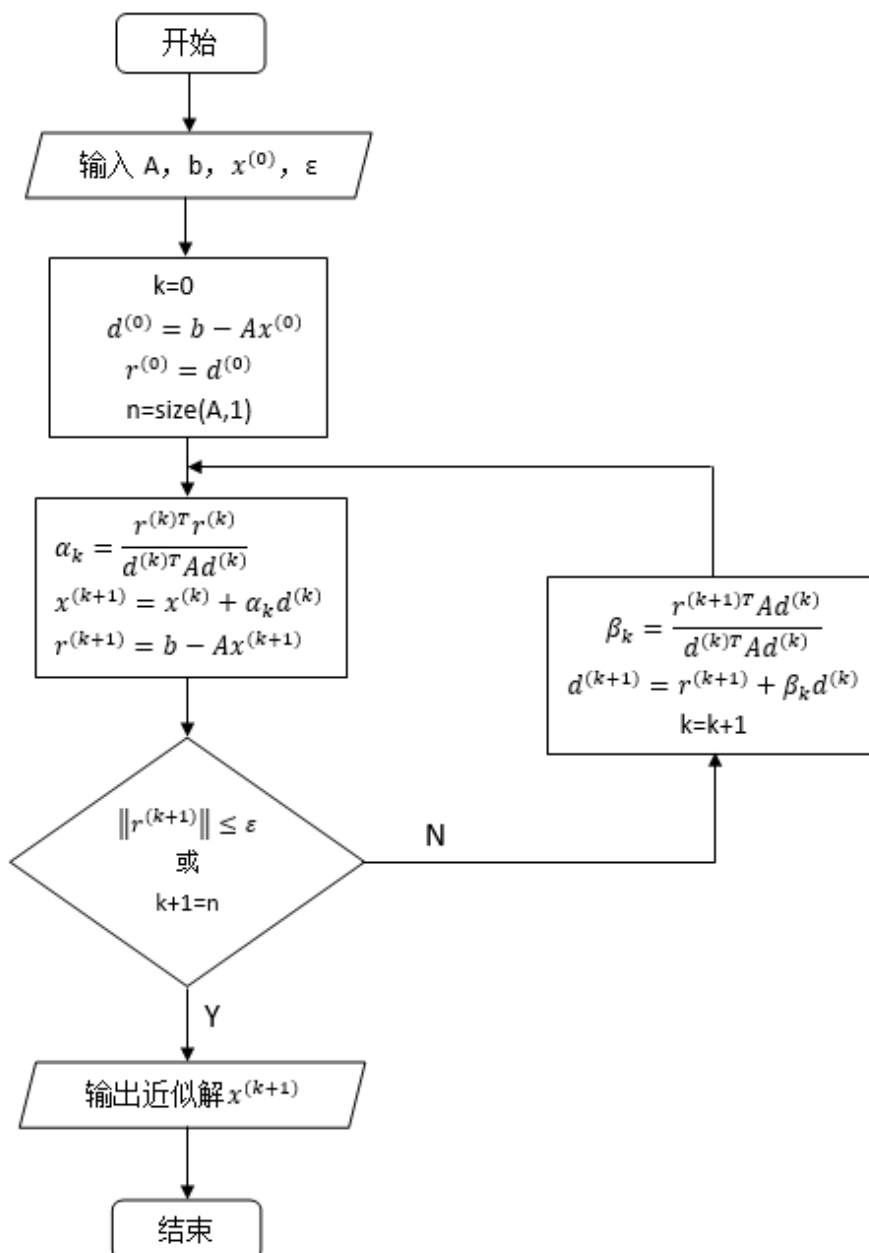
$$(v) \beta_k = \frac{\|r^{(k+1)}\|_2^2}{\|r^{(k)}\|_2^2};$$

$$(vi) d^{(k+1)} = r^{(k+1)} + \beta_k d^{(k)};$$

end do

计算经验表明，对于不是十分病态的问题，共轭梯度法收敛较快，迭代次数远小于矩阵的阶数 n 。对于病态问题，只要进行足够多次的迭代（迭代次数大约为矩阵阶数 n 的 3-5 倍）后，一般也能得到满意的结果，因而共轭梯度法是求解高阶稀疏线性方程组的一个有效、常用的方法。

二、 程序框图

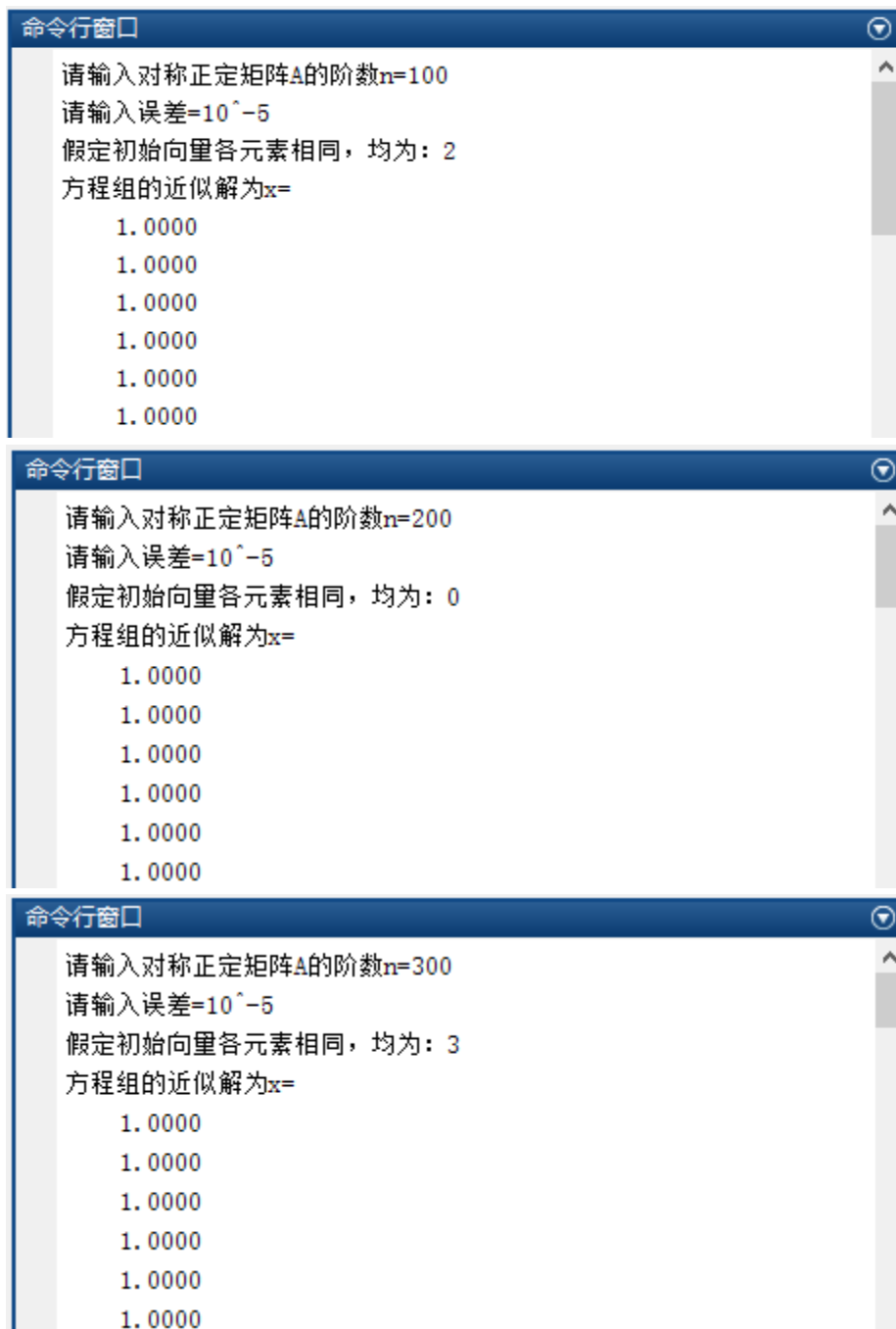


三、程序使用说明及案例计算结果

程序使用说明

本共轭梯度法求解线性方程的程序需要用户给定对称正定矩阵 A 的阶数，误差以及初始向量，然后程序自动调用定义的函数 `Gongetidu(A,b,x0,epsa)` 实现求解线性方程组 $Ax=b$ 。其中 A ， b 的阶数，初始向量 x_0 和 $epsa$ 都是可变的。

案例计算结果



可以发现，当 $n=100, 200, 300$ 时，方程组 $Ax=b$ 的解 x 为元素均为 1 的 n 阶向量。

四、Matlab 程序（M-文件）

```
clear A b x0;clc; %程序运行前首先清除 A, b 和 x0 的数值, 以免影响计算
n=input('请输入对称正定矩阵 A 的阶数 n=');
epsa=input('请输入误差=');
A=zeros(0,0); %给矩阵 A 预先配置内存空间, 减少耗时
for k=1:(n-1) %读取题目 3.2 的矩阵 A
    A(k,k)=-2;
    A(k,k+1)=1;
    A(k+1,k)=1;
end
A(n,n)=-2;
A;
b(1,1)=-1; %读取题目 3.2 的矩阵 b
b(n,1)=-1;
b;
x0(1,1)=input('假定初始向量各元素相同, 均为: '); %给题目 3.2 的迭代过程赋初值
for kk=2:n
    x0(kk,1)=x0(1,1);
end
x=Gongetidu(A,b,x0,epsa); %调用共轭梯度法求线性方程组的函数
function x=Gongetidu(A,b,x0,epsa)
n=size(A,1);
x=x0;
r=b-A*x;
d=r;
for k=0:(n-1)
    alpha=(r'*r)/(d'*A*d);
    x=x+alpha*d;
    r2=b-A*x;
    if ((norm(r2)<=epsa) || (k==n-1))
        disp('方程组的近似解为 x=');
        disp(x);
        break;
    end
    beta=norm(r2)^2/norm(r)^2;
    d=r2+beta*d;
    r=r2;
end
end
```

三次样条插值

三、 算法原理

分段低次插值多项式的一阶或二阶导数不连续，不能满足许多实际问题的要求。例如，在飞机机翼外形设计，船体放样等许多实际问题中，要求曲线二阶光滑，即要求函数的二阶导数连续。为了将一些已知点连成一条充分光滑的曲线，早期的做法是绘图人员用压铁把一条称为样条的富有弹性的细木条或金属条固定在相邻的若干点上，然后沿样条画下一条所需的光滑曲线，这条曲线称为样条曲线。实际上，它是由分段三次曲线拼接而成的，在连接点处二阶导数连续。将样条曲线抽象为数学问题称为样条函数。

1、三次样条插值函数的定义

设在区间 $[a,b]$ 上给定 $n+1$ 个节点 $x_i(a \leq x_0 < x_1 < \dots < x_n \leq b)$ ，在节点 x_i 处的函数值 $y_i = f(x_i)(i = 0, 1, \dots, n)$ 。若函数 $S(x)$ 满足以下三条：

- (1) 在每个子区间 $[x_{i-1}, x_i]$ 上， $S(x)$ 是三次多项式；
- (2) 是个 $S(x_i) = y_i(i = 0, 1, \dots, n)$ ；
- (3) 在区间 $[a,b]$ 上， $S(x)$ 的二阶导数 $S''(x)$ 连续，

则称 $S(x)$ 为函数 $y = f(x)$ 在区间 $[a,b]$ 上的三次样条插值函数。

由定义可知， $S(x)$ 是分段三次多项式，故在每个子区间 $[x_{i-1}, x_i](i = 1, \dots, n)$ 上， $S(x)$ 有 4 个待定参数，共有 n 个子区间，所以 $S(x)$ 共有 $4n$ 个待定参数。根据定义中的条件 (3) 知

$$\begin{cases} S_-(x_i) = S_+(x_i), \\ \dot{S}_-(x_i) = \dot{S}_+(x_i), \quad i = 1, 2, \dots, n-1. \\ \ddot{S}_-(x_i) = \ddot{S}_+(x_i), \end{cases}$$

上式共有 $3n-3$ 个条件，再加上定义条件 (2) 中的 $n+1$ 个条件，共有 $4n-2$ 个条件。还需要增加两个条件才能确定 $4n$ 个待定参数，即才能确定 $S(x)$ 。所增加的条件称为边界条件或端点条件。三种常用的边界条件如下：

- (1) 已知 $f(x)$ 在区间 $[a,b]$ 的两端点 a, b 处的二阶导数值 $f''(a), f''(b)$ ；
- (2) 已知 $f(x)$ 在区间 $[a,b]$ 的两端点 a, b 处的一阶导数值 $f'(a), f'(b)$ ；
- (3) 已知函数 $f(x)$ 式以 $T=b-a$ 为周期的周期函数。

2、三次样条插值函数的导出

1) 导出在子区间 $[x_{i-1}, x_i](i = 1, 2, \dots, n)$ 上的 $S(x)$ 的表达式

由于 $S(x)$ 的二阶导数连续，设 $S(x)$ 在节点 x_i 处的二阶导数值 $S''(x_i) = M_i(i = 1, 2, \dots, n)$ ，其中 M_i 为未知的待定参数。由 $S(x)$ 是分段的三次多项式知， $S''(x)$ 是分段线性函数， $S''(x)$ 在子区间 $[x_{i-1}, x_i]$ 上可表示为

$$\begin{aligned}
S''(x) &= \frac{x - x_i}{x_{i-1} - x_i} M_{i-1} + \frac{x - x_{i-1}}{x_i - x_{i-1}} M_i \\
&= \frac{x_i - x}{h_i} M_{i-1} + \frac{x - x_{i-1}}{h_i} M_i \quad x_{i-1} \leq x \leq x_i
\end{aligned}$$

其中 $h_i = x_i - x_{i-1}$ 。对上式两端积分两次得

$$\begin{aligned}
S(x) &= \frac{(x_i - x)^3}{6h_i} M_{i-1} + \frac{(x - x_{i-1})^3}{6h_i} M_i \\
&\quad + b_i(x_i - x) + c_i(x - x_{i-1}) \quad x_{i-1} \leq x \leq x_i
\end{aligned}$$

其中 b_i, c_i 为积分常数。现在确定 b_i, c_i 。由插值条件 $S(x_{i-1}) = y_{i-1}, S(x_i) = y_i$ 得

$$\frac{h_i^2}{6} M_{i-1} + b_i h_i = y_{i-1}, \quad \frac{h_i^2}{6} M_i + b_i h_i = y_i.$$

由此解得

$$\begin{aligned}
b_i &= (y_{i-1} - \frac{h_i^2}{6} M_{i-1}) / h_i \\
c_i &= (y_i - \frac{h_i^2}{6} M_i) / h_i
\end{aligned}$$

将 b_i, c_i 代入 $S(x)$ 可得 $S(x)$ 在子区间 $[x_{i-1}, x_i] (i = 1, 2, \dots, n)$ 上的表达式为

$$\begin{aligned}
S(x) &= \frac{(x_i - x)^3}{6h_i} M_{i-1} + \frac{(x - x_{i-1})^3}{6h_i} M_i + (y_{i-1} - \frac{h_i^2}{6} M_{i-1}) \frac{x_i - x}{h_i} \\
&\quad + \left(y_i - \frac{h_i^2}{6} M_i \right) \frac{x - x_{i-1}}{h_i} \quad x_{i-1} \leq x \leq x_i
\end{aligned}$$

2) 建立参数 M_i 的方程组

对 $S(x)$ 求导可得

$$\begin{aligned}
S'(x) &= \frac{(x_i - x)^2}{2h_i} M_{i-1} + \frac{(x - x_{i-1})^2}{2h_i} M_i \\
&\quad + \frac{y_i - y_{i-1}}{h_i} - \frac{M_i - M_{i-1}}{6} h_i \quad x_{i-1} \leq x \leq x_i
\end{aligned}$$

上式中令 $x = x_i$ 得 $S(x)$ 在 x_i 处的左导数

$$\begin{aligned}
S'_-(x_i) &= \frac{h_i}{2} M_i + \frac{y_i - y_{i-1}}{h_i} - \frac{M_i - M_{i-1}}{6} h_i \\
&= \frac{h_i}{6} M_{i-1} + \frac{h_i}{3} M_i + \frac{y_i - y_{i-1}}{h_i}
\end{aligned}$$

令 $x = x_{i-1}$ 得 $S(x)$ 在 x_{i-1} 处的右导数

$$\begin{aligned}
S'_+(x_{i-1}) &= -\frac{h_i}{2} M_{i-1} + \frac{y_i - y_{i-1}}{h_i} - \frac{M_i - M_{i-1}}{6} h_i \\
&= -\frac{h_i}{3} M_{i-1} - \frac{h_i}{6} M_i + \frac{y_i - y_{i-1}}{h_i}
\end{aligned}$$

从而

$$\dot{S}_+(x_i) = -\frac{h_{i+1}}{3}M_i - \frac{h_{i+1}}{6}M_{i+1} + \frac{y_{i+1} - y_i}{h_{i+1}}$$

由 $S(x)$ 在内节点 x_i 处一阶导数的连续性可知

$$\dot{S}_-(x_i) = \dot{S}_+(x_i) \quad i = 1, 2, \dots, n-1.$$

即

$$\frac{h_i}{6}M_{i-1} + \frac{h_i + h_{i+1}}{3}M_i + \frac{h_{i+1}}{6}M_{i+1} = \frac{y_{i+1} - y_i}{h_{i+1}} - \frac{y_i - y_{i-1}}{h_i} \quad i = 1, 2, \dots, n-1.$$

上式两端同乘 $\frac{6}{h_i + h_{i+1}}$ 得

$$\frac{h_i}{h_i + h_{i+1}}M_{i-1} + 2M_i + \frac{h_{i+1}}{h_i + h_{i+1}}M_{i+1} = \frac{6}{h_i + h_{i+1}}\left(\frac{y_{i+1} - y_i}{h_{i+1}} - \frac{y_i - y_{i-1}}{h_i}\right)$$

记

$$\begin{aligned} \mu_i &= \frac{h_i}{h_i + h_{i+1}} & \lambda_i &= \frac{h_{i+1}}{h_i + h_{i+1}} = 1 - \mu_i \\ d_i &= \frac{6}{h_i + h_{i+1}}\left(\frac{y_{i+1} - y_i}{h_{i+1}} - \frac{y_i - y_{i-1}}{h_i}\right) = 6f[x_{i-1}, x_i, x_{i+1}] \quad i = 1, 2, \dots, n-1. \end{aligned}$$

则关于 M_i 的方程组可写为

$$\mu_i M_{i-1} + 2M_i + \lambda_i M_{i+1} = d_i \quad i = 1, 2, \dots, n-1.$$

上式中的未知量 M_i 在力学上解释为细梁在 x_i 截面处的弯矩，故称为三弯矩方程组。三弯矩方程组中只有 $n-1$ 个方程，不能完全确定 $n+1$ 个未知量 $M_i (i = 0, 1, 2, \dots, n)$ ，所以欲解三弯矩方程组或者减少两个未知量，或者再增加两个方程，这由边界条件来确定。

3) 三种边界条件的三弯矩方程

(1) 第一种边界条件：已知 $f(\ddot{a}), f(\ddot{b})$ 。取 $M_0 = f(\ddot{a}), M_n = f(\ddot{b})$ ，这时三弯矩方程组减少了两个未知量，变成只含 $n-1$ 个未知量 $M_i (i = 1, 2, \dots, n-1)$ 的 $n-1$ 个方程的方程组

$$\begin{cases} 2M_1 + \lambda_1 M_2 = d_1 - \mu_0 M_0 \\ \mu_i M_{i-1} + 2M_i + \lambda_i M_{i+1} = d_i & i = 2, 3, \dots, n-2. \\ \mu_{n-1} M_{n-2} + 2M_{n-1} = d_{n-1} - \lambda_{n-1} M_n \end{cases} \quad (1)$$

用矩阵表示为

$$\begin{pmatrix} 2 & \lambda_1 & & & \\ \mu_2 & 2 & \lambda_2 & & \\ & \ddots & \ddots & \ddots & \\ & & \mu_{n-2} & 2 & \lambda_{n-2} \\ & & & \mu_{n-1} & 2 \end{pmatrix} \begin{pmatrix} M_1 \\ M_2 \\ \vdots \\ M_{n-2} \\ M_{n-1} \end{pmatrix} = \begin{pmatrix} d_1 - \mu_1 M_0 \\ d_2 \\ \vdots \\ d_{n-2} \\ d_{n-1} - \lambda_{n-1} M_n \end{pmatrix}$$

注意到 $\lambda_i + \mu_i = 1 < 2$ ，以上方程组系数矩阵是严格三对角占优矩阵，可用追赶法求解。解出 $M_i (i = 1, 2, \dots, n-1)$ 后，代入 $S(x)$ 即可得三次样条插值函数的数学表达式。

(2) 第二种边界条件：已知 $f(a), f(b)$ 。记 $y_0 = f(a), y_n = f(b)$ ，则有 $S_+(x_0) = y_0, S_-(x_n) = y_n$ 。故有

$$-\frac{h_1}{3}M_0 - \frac{h_1}{6}M_1 + \frac{y_1 - y_0}{h_1} = y_0 \quad \frac{h_n}{6}M_{n-1} + \frac{h_n}{3}M_n + \frac{y_n - y_{n-1}}{h_n} = y_n$$

即

$$2M_0 + M_1 = d_0 \quad M_{n-1} + 2M_n = d_n$$

其中

$$d_0 = \frac{6}{h_1} \left(\frac{y_1 - y_0}{h_1} - y_0 \right) = 6f[x_0, x_0, x_1]$$

$$d_n = \frac{6}{h_n} \left(y_n - \frac{y_n - y_{n-1}}{h_n} \right) = 6f[x_{n-1}, x_n, x_n]$$

将以上两个方程添加进方程组中，得到关于第二种边界条件的三弯矩方程组

$$\begin{cases} 2M_0 + M_1 = d_0 \\ \mu_i M_{i-1} + 2M_i + \lambda_i M_{i+1} = d_i \quad i = 1, 2, 3, \dots, n-1. \\ M_{n-1} + 2M_n = d_n \end{cases} \quad (2)$$

用矩阵表示为

$$\begin{pmatrix} 2 & 1 & & & & \\ \mu_1 & 2 & \lambda_1 & & & \\ & \mu_2 & 2 & \lambda_2 & & \\ & & \ddots & \ddots & \ddots & \\ & & & \mu_{n-1} & 2 & \lambda_{n-1} \\ & & & & 1 & 2 \end{pmatrix} \begin{pmatrix} M_0 \\ M_1 \\ M_2 \\ \vdots \\ M_{n-1} \\ M_n \end{pmatrix} = \begin{pmatrix} d_0 \\ d_1 \\ d_2 \\ \vdots \\ d_{n-1} \\ d_n \end{pmatrix}$$

同样，该方程的系数矩阵是严格三对角占优矩阵，可用追赶法求解。

(3) 第三种边界条件：周期型边界条件。已知 $y=f(x)$ 是以 $T = b - a = x_n - x_0$ 为周期的周期函数，则由周期性可知 $y_n = y_0, y_{n+1} = y_1, M_n = M_0, M_{n+1} = M_1, h_{n+1} = h_1$ 。这时，将点 x_n 看成内节点，则方程组对 $i=n$ 也成立，即有

$$\mu_n M_{n-1} + 2M_n + \lambda_n M_{n+1} = d_n$$

也即

$$\lambda_n M_1 + \mu_n M_{n-1} + 2M_n = d_n$$

其中

$$d_n = \frac{6}{h_n + h_{n+1}} \left(\frac{y_{n+1} - y_n}{h_{n+1}} - \frac{y_n - y_{n-1}}{h_n} \right) = \frac{6}{h_n + h_1} \left(\frac{y_1 - y_n}{h_1} - \frac{y_n - y_{n-1}}{h_n} \right)$$

方程组的第 1 个 ($i=1$) 方程为

$$\mu_1 M_0 + 2M_1 + \lambda_1 M_2 = d_1$$

即

$$2M_1 + \lambda_1 M_2 + \mu_1 M_n = d_1$$

于是可得关于第三种边界条件的三弯矩方程组

$$\begin{cases} 2M_1 + \lambda_1 M_2 + \mu_1 M_n = d_1 \\ \mu_i M_{i-1} + 2M_i + \lambda_i M_{i+1} = d_i \quad i = 2, 3, \dots, n-1, \\ \lambda_n M_1 + \mu_n M_{n-1} + 2M_n = d_n \end{cases} \quad (3)$$

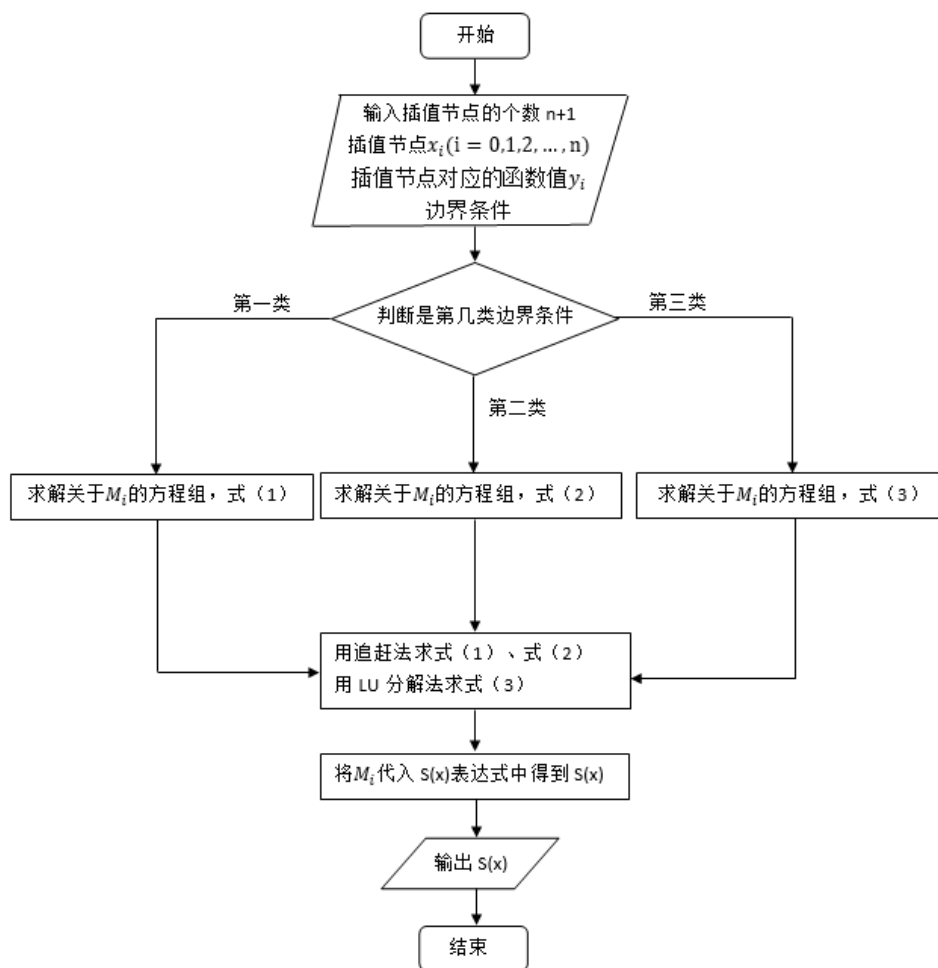
用矩阵表示为

$$\begin{pmatrix} 2 & \lambda_1 & & & \mu_1 \\ \mu_2 & 2 & \lambda_2 & & \\ & \ddots & \ddots & \ddots & \\ & & \mu_{n-1} & 2 & \lambda_{n-1} \\ \lambda_n & & & \mu_n & 2 \end{pmatrix} \begin{pmatrix} M_1 \\ M_2 \\ \vdots \\ M_{n-1} \\ M_n \end{pmatrix} = \begin{pmatrix} d_1 \\ d_2 \\ \vdots \\ d_{n-1} \\ d_n \end{pmatrix}$$

显然，该方程的系数矩阵也是严格对角占优矩阵，可用 LU 方法求解。

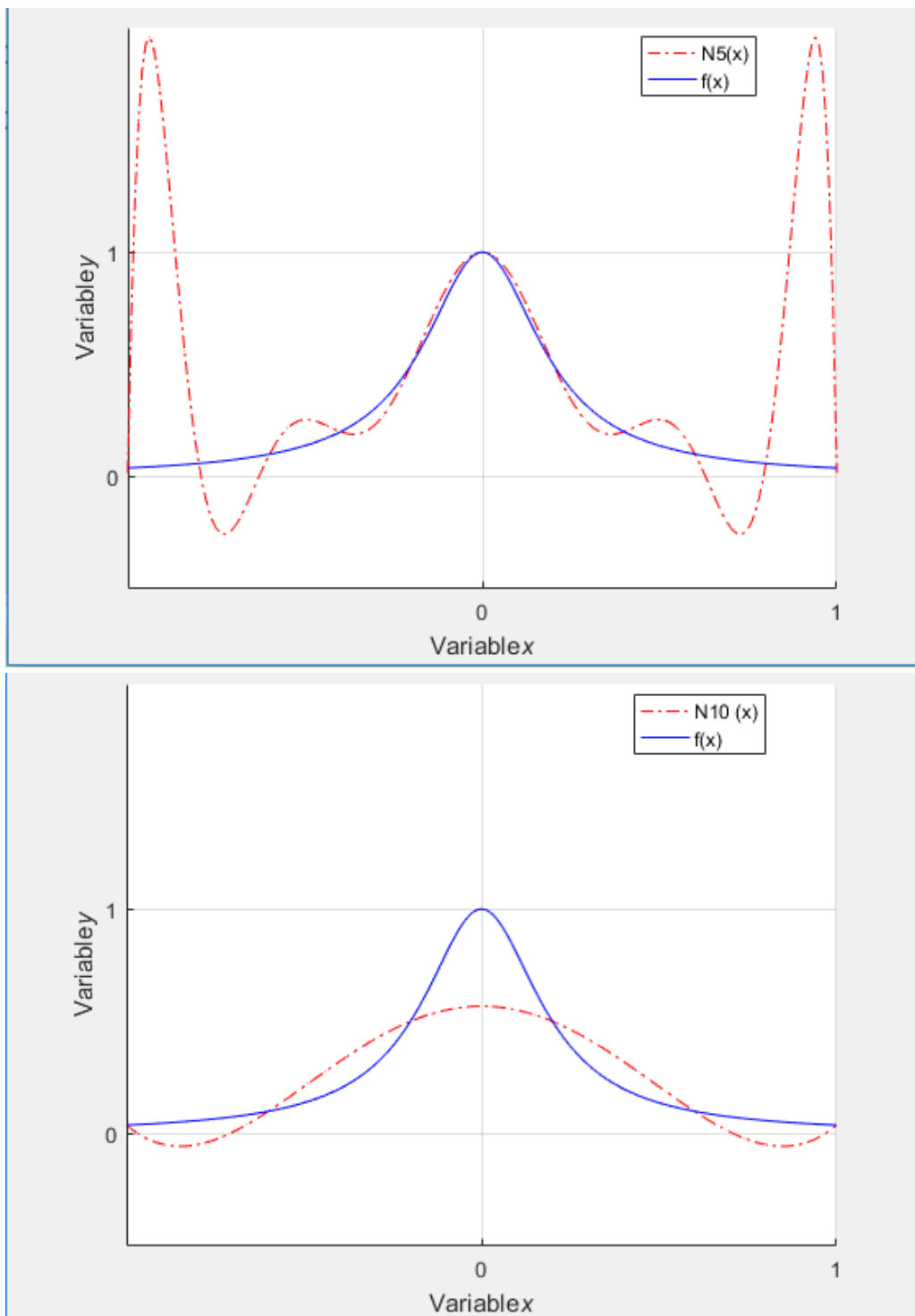
根据不同的边界条件，求解相应的三弯矩方程组，将求得的 M_i 代入 $S(x)$ 的表达式便可得三次样条插值函数 $S(x)$ 。

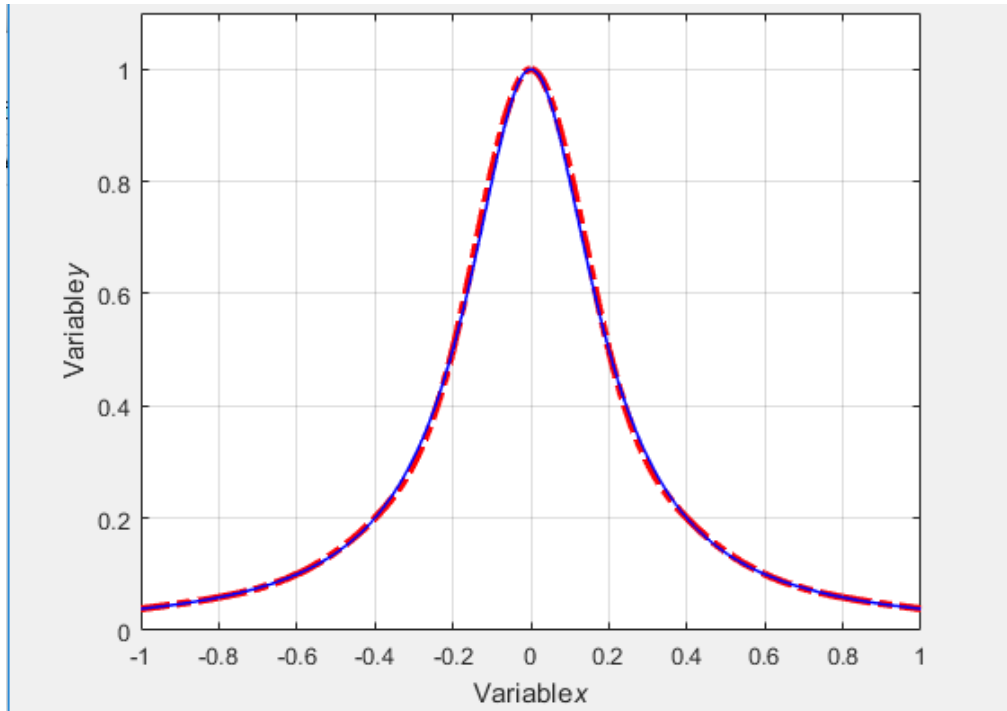
四、 程序框图



三、程序使用说明及案例计算结果

改程序不需要用户输入数据，可直接计算 $N_5(x)$ 、 $N_{10}(x)$ 、 $S_{10}(x)$ ，并将其分别与原函数 $f(x) = \frac{1}{1+25x^2}$ 画在同一坐标系内，可以清楚的观察到 $N_5(x)$ 的拟合效果没有 $N_{10}(x)$ 好，而 $S_{10}(x)$ 的和原函数基本重合，误差最小。程序运行结果如下：





四、Matlab 程序（M-文件）

改程序分为主程序、threetimes 函数、newtonway 函数三部分，分别如下：

主程序

```
clear;clc;
threetimes(10) %调用三次样条插值 段数为 10
hold on
newtonway(5) %调用牛顿插值法 段数为 5
hold on
newtonway(10) %调用牛顿插值法 段数为 10
hold on
```

threetimes 函数

```
function threetimes(n)
k=1;syms t;
y=1/(1+25*k^.2); %测试函数公式
%区间段数，由函数传递值确定
h=2/n; %区间段长
cout=1; %cout 为计数器
x=[]; %x 轴节点数
for i=-1:h:1 %计算节点数值
k=i;
x(cout)=i;
y(cout)=1/(1+25*k^2);
```

```

    cout=cout+1;
end
% x=[-3 -1 0 3 4];
% y=[7 11 26 56 29];
for i=1:1:size(x,2) %b 插商矩阵 前两列赋值
    b(i,1)=x(i);
    b(i,2)=y(i);
end
f=1; %计数变量
for j=3:1:size(x,2)+1 %计算插商循环 size(x,2)区间节点个数
    k=1;
    for i=j-1:1:size(x,2)
        b(i,j)=(b(i,j-1)-b(i-1,j-1))/(b(i,1)-b(i+2-j,1));
        if(k==1)
            c(f)=b(i,j);%提取插商系数
            k=0;
            f=f+1;
        end
    end
end
end
for i=3:1:size(x,2) %d 等于 6 倍于 三阶插商
    d(i-2)=6*b(i,4); %d 为求解 m 矩阵的最后一侧列向量
end
for i=2:1:size(x,2)
    h(i-1)=b(i,1)-b(i-1,1);
end
for i=1:1:size(h,2)-1
    u(i)=h(i)./(h(i)+h(i+1));
    r(i)=1-u(i);
end
for i=1:1:size(x,2)-3
    bb(i,i)=2; %对角阵数值
    bb(i,i+1)=r(i); %对角数上面的值
    bb(i+1,i)=u(i+1); %对角下面的数值
end
bb(size(x,2)-2,size(x,2)-2)=2;
mm=bb\d';
m=[]; % M 矩阵 M (0) 和 M (n) =0;
m(1)=0;

```

```

for i=1:1:size(mm,1)
    m(i+1)=mm(i);
end
m(size(m,2)+1)=0;
subplot()
for i=2:1:size(x,2)      % 通过公式确定每段拟合函数
    s_1=m(i-1)*(x(i)-t).^3/(6*h(i-1)); % 注意其中下标顺序
    s_2=m(i)*(t-x(i-1)).^3/(6*h(i-1));
    s_3=(y(i-1)-h(i-1).^2*m(i-1)/6)*(x(i)-t)/h(i-1);
    s_4=(y(i)-h(i-1).^2*m(i)/6)*(t-x(i-1))/h(i-1);
    s=s_1+s_2+s_3+s_4;
    s=expand(s);
    a=x(i-1):0.01:x(i);
    f=subs(s,t,a);
    plot(a,f,'r-','linewidth',3)
    hold on
end
x=-1:0.01:1;
axis([-1 1 0 1.1]);
y1=power(1+25*power(x,2),-1);
plot(x,y1,'b-','linewidth',1)
xlabel('Variable\it{x}')
ylabel('Variable\it{y}')
grid
hold on
end
newtonway 函数
function newtonway(n)
k=1;syms t;
y=1/(1+25*k.^2); %公式
    %区间段数
h=2/n;      %区间短长
cout=1;
x=[];
for i=-1:h:1
    k=i;
    x(cout)=i;
    y(cout)=1/(1+25*k^2); %计算节点数值
    cout=cout+1;
end

```



```

end
size(x,2);
b=[size(x,2),n];
for i=1:1:size(x,2) %b 插商矩阵赋值
    b(i,1)=x(i);
    b(i,2)=y(i);
end
f=1; %计数变量
for j=3:1:size(x,2)+1 %计算插商循环
    m=1;
    for i=j-1:1:n+1
        b(i,j)=(b(i,j-1)-b(i-1,j-1))/(b(i,1)-b(i+2-j,1));
        if(m==1)
            c(f)=b(i,j);%提取插商系数
            m=0;
            f=f+1;
        end
    end
end
end
b;
l=1;d=b(1,2);d;
for i=1:1:size(x,2)-1 %计算出多项式
    l=l*(t-b(i,1));
    d=d+c(i)*l;
end
d=simplify(d);
s=2; %设定画图区间上下界数值
t=-s:0.01:s;
k=-s:0.01:s;
y=1./(1+25.*k.*k);
figure;
axis([-1,1,-0.5,2]);
set(gca,'XTickMode','manual','XTick',[0 1 2 3 4 5 6 7 8 9 10]);
set(gca,'YTickMode','manual','YTick',[-1 0 1]);grid
xlabel('Variable\it{x}')
ylabel('Variable\it{y}')
hold on
plot(t,subs(d),'r-.',k,y,'b')
end

```

龙格贝积分

五、 算法原理

对于复化梯形求积公式而言，可取积分的近似值为

$$I(f) \approx T_{2n} + \frac{1}{4-1}(T_{2n} - T_n) = \bar{T}_{2n} \quad (1)$$

对于复化辛普森求积公式有

$$I(f) - S_n = -\frac{b-a}{2880}h^4 f^{(4)}(\eta) \quad a \leq \eta \leq b$$

$$I(f) - S_{2n} = -\frac{b-a}{2880}\left(\frac{h}{2}\right)^4 f^{(4)}(\eta_1) \quad a \leq \eta_1 \leq b$$

假定 $f^{(4)}(x)$ 在区间 $[a,b]$ 上连续且变化不大，可以认为 $f^{(4)}(\eta) \approx f^{(4)}(\eta_1)$ 。将以上两式相除得

$$\frac{I(f) - S_n}{I(f) - S_{2n}} \approx 16$$

由此解得

$$I(f) \approx S_{2n} + \frac{1}{4^2-1}(S_{2n} - S_n) \triangleq \bar{S}_{2n} \quad (2)$$

同理，对复化科茨求积公式有

$$I(f) \approx C_{2n} + \frac{1}{4^3-1}(C_{2n} - C_n) \triangleq \bar{C}_{2n} \quad (3)$$

下面对(1)、(2)、(3)作进一步分析，看一看它们的实质和它们之间的关系。

$$\begin{aligned} \bar{T}_{2n} &= T_{2n} + \frac{1}{3}(T_{2n} - T_n) = \frac{1}{3}(4T_{2n} - T_n) \\ &= \frac{1}{3} \left\{ 4 \left[\frac{1}{2}T_n + \frac{h}{2} \sum_{i=1}^n f\left(\frac{x_{i-1} + x_i}{2}\right) \right] - T_n \right\} \\ &= \frac{1}{3} \left\{ T_n + 2h \sum_{i=1}^n f\left(\frac{x_{i-1} + x_i}{2}\right) \right\} \\ &= \frac{1}{3} \left\{ \frac{h}{2} \left[f(a) + 2 \sum_{i=1}^{n-1} f(x_i) + f(b) \right] + 2h \sum_{i=1}^n f\left(\frac{x_{i-1} + x_i}{2}\right) \right\} \\ &= \frac{1}{6} \left[f(a) + 2 \sum_{i=1}^{n-1} f(x_i) + 4 \sum_{i=1}^n f\left(\frac{x_{i-1} + x_i}{2}\right) + f(b) \right] = S_n \end{aligned}$$

即

$$S_n = T_{2n} + \frac{1}{4-1}(T_{2n} - T_n)$$

同理可得

$$C_n = S_{2n} + \frac{1}{4^2 - 1} (S_{2n} - S_n)$$

令

$$R_n = C_{2n} + \frac{1}{4^3 - 1} (C_{2n} - C_n) \quad (4)$$

(4)式即称为龙格贝积分公式。其截断误差为 $c_R h^8 f^{(8)}(\eta)$ 。以此类推可知，若取

$$R_{2n} + \frac{1}{4^4 - 1} (R_{2n} - R_n)$$

作为积分的近似值其截断误差是 $ch^{10}f^{(10)}(\eta)$ 。容易设想取其作为积分的近似值可能更精确。事实上并非完全如此，这是因为被积函数 $f(x)$ 的高阶导数性态很难估计，特别是高阶导数性态不好的函数， $f^{(10)}(x)$ 的数量级可能很大，虽然 h^{10} 很小，但截断误差项 $ch^{10}f^{(10)}(\eta)$ 可能很大。如此计算的效果可能并不理想。再者，一般地， $R_{2n} - R_n$ 是两个相近的数相减，可能产生较大的误差。因此，一般不再继续进行外推，计算到龙格贝积分公式就已经足够准确。如果积分区间 $[a,b]$ 较大，则最好事先把 $[a,b]$ 分成若干个子区间，然后在每个子区间上再进行数值积分。

龙格贝积分法是将区间 $[a,b]$ 逐次分半，逐次递推计算，再用(1)~(4)逐次计算，以得到较高精度的积分近似值。龙格贝积分法的计算公式如下：

$$T_1 = \frac{b-a}{2} [f(a) + f(b)]$$

$$T_{2^{k+1}} = \frac{1}{2} T_{2^k} + \frac{b-a}{2^{k+1}} \sum_{i=1}^{2^k} f(a + (2i-1) \frac{b-a}{2^{k+1}}) \quad k = 0, 1, 2, \dots,$$

$$S_{2^k} = T_{2^{k+1}} + \frac{1}{4-1} (T_{2^{k+1}} - T_{2^k})$$

$$C_{2^k} = S_{2^{k+1}} + \frac{1}{4^2-1} (S_{2^{k+1}} - S_{2^k})$$

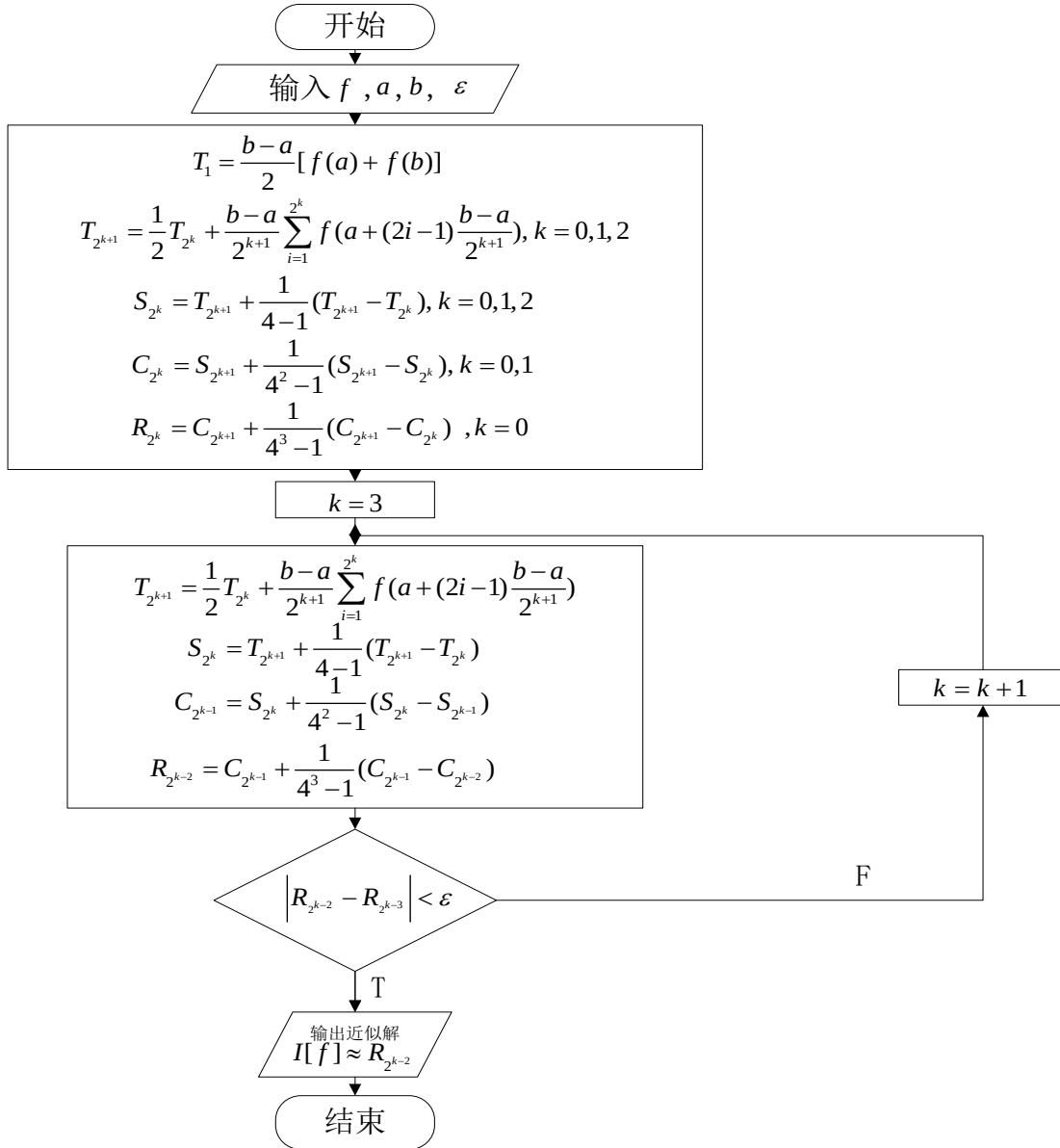
$$R_{2^k} = C_{2^{k+1}} + \frac{1}{4^3-1} (C_{2^{k+1}} - C_{2^k})$$

计算过程如下图所示进行。

T		S		C		R
T_1						
	$\searrow$					
T_2	$\rightarrow$	S_1				
	$\searrow$		$\searrow$			
T_{2^2}	$\rightarrow$	S_2	$\rightarrow$	C_1		
	$\searrow$		$\searrow$		$\searrow$	
T_{2^3}	$\rightarrow$	S_{2^2}	$\rightarrow$	C_2	$\rightarrow$	R_1
	$\searrow$		$\searrow$		$\searrow$	
T_{2^4}	$\rightarrow$	S_{2^3}	$\rightarrow$	C_{2^2}	$\rightarrow$	R_2
$\vdots$		$\vdots$		$\vdots$		$\vdots$

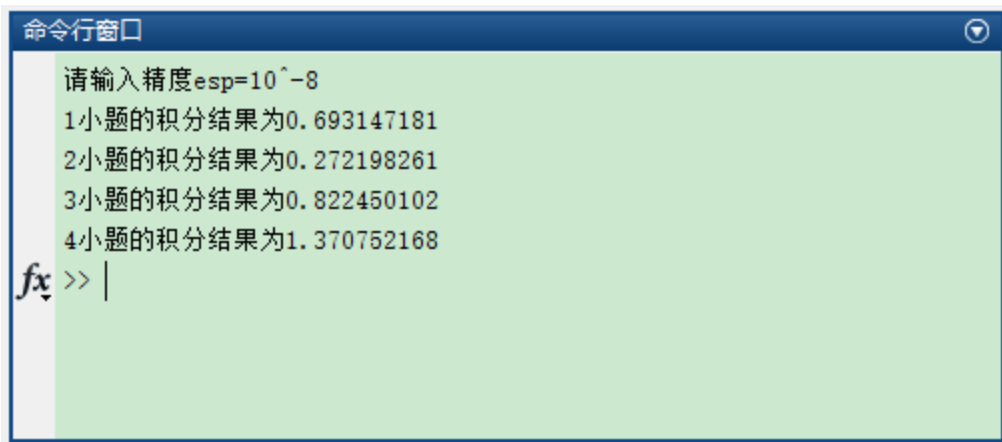
若 $|R_{2^{k+1}} - R_{2^k}| < \varepsilon$ ，则取 $I[f] \approx R_{2^{k+1}}$ ；否则继续计算，直到满足精度要求为止。

六、程序框图



三、程序使用说明及案例计算结果

改程序只需要输入精度要求即可求解习题 6.2 中的 4 个小题，其中第 3、第 4 个小题的函数边界值比较特殊，故进行近似化处理。程序运行结果如下：

A screenshot of the MATLAB Command Window. The title bar is blue with the text '命令行窗口' (Command Window) and a close button. The background is light green. The text inside shows a prompt '请输入精度 esp=10^-8' followed by four lines of results: '1小题的积分结果为0.693147181', '2小题的积分结果为0.272198261', '3小题的积分结果为0.822450102', and '4小题的积分结果为1.370752168'. At the bottom, there is a prompt 'fx >> |' with a cursor.

```
命令行窗口
请输入精度 esp=10^-8
1小题的积分结果为0.693147181
2小题的积分结果为0.272198261
3小题的积分结果为0.822450102
4小题的积分结果为1.370752168
fx >> |
```

四、Matlab 程序（M-文件）

```
clear;clc;
esp=input('请输入精度 esp=');
for i=1:4
    if i<3
        y=Romberg(0,1,i,esp);
        fprintf('%0f 小题的积分结果为%.9f\n',i,y);
    end
    if i==3
        y=Romberg(0.00001,0.99999,i,esp); %由于某些函数边界值比较特殊 故进行近似化
        fprintf('%0f 小题的积分结果为%.9f\n',i,y);
    end
    if i==4
        y=Romberg(0.00001,pi/2,i,esp);
        fprintf('%0f 小题的积分结果为%.9f\n',i,y);
    end
end

function[y]=f(x,n)
if n==1
    y=1/(1+x);
elseif n==2
    y=log(1+x)/(1+x^2);
elseif n==3
    y=log(1+x)/x;
else
    y=sin(x)/x;
end
end
```

```

function[y]=T(k,a,b,n)
t(1)=(b-a)/2*(f(a,n)+f(b,n));
if k==1
    y=t(1);
else
    sum=0;
    for i=1:k/2
        sum=sum+f(a+(2*i-1)*(b-a)/k,n);
    end
    y=1/2*T(k/2,a,b,n)+(b-a)/k*sum;
end
end

```

```

function[y]=S(k,a,b,n)
y=T(2*k,a,b,n)+1/(4-1)*(T(2*k,a,b,n)-T(k,a,b,n));
end

```

```

function[y]=C(k,a,b,n)
y=S(2*k,a,b,n)+1/(4^2-1)*(S(2*k,a,b,n)-S(k,a,b,n));
end

```

```

function[y]=Romberg(a,b,n,eps)
eps1=10;
k=1;
R(1)=C(2,a,b,n)+1/(4^3-1)*(C(2,a,b,n)-C(1,a,b,n));
while eps1>=eps
    R(2^k)=C(2^(k+1),a,b,n)+1/(4^3-1)*(C(2^(k+1),a,b,n)-C(2^k,a,b,n));
    eps1=abs(R(2^k)-R(2^(k-1)));
    k=k+1;
end
y=R(2^(k-1));
end

```

四阶龙格-库塔法

七、 算法原理

利用泰勒展开可以导出龙格-库塔法（简称 R-K 方法）。m 级的龙格-库塔法的一般形式为

$$\begin{cases} y_{i+1} = y_i + \lambda_1 K_1 + \lambda_2 K_2 + \cdots + \lambda_m K_m \\ K_1 = hf(x_i, y_i) \\ K_2 = hf(x_i + \alpha_2 h, y_i + \beta_{21} K_1) \\ K_3 = hf(x_i + \alpha_3 h, y_i + \beta_{31} K_1 + \beta_{32} K_2) \\ \quad \dots \dots \\ K_m = hf(x_i + \alpha_m h, y_i + \beta_{m1} K_1 + \beta_{m2} K_2 + \cdots + \beta_{m, m-1} K_{m-1}) \end{cases}$$

其中 $\lambda_i, \alpha_i, \beta_{j,k}$ 均为常数，有待定系数法确定。确定的原则是将局部截断误差 $R[y] = y(x_{i+1}) - y_{i+1}$ 在处 x_i 泰勒展开，适当选取 h 的系数，使得局部截断误差 $R[y]$ 的阶数尽可能高。

下面以 m=2 级的 R-K 法为例来说明 R-K 法的导出思想。2 级 R-K 法形式如下：

$$\begin{cases} y_{i+1} = y_i + \lambda_1 K_1 + \lambda_2 K_2 \\ K_1 = hf(x_i, y_i) \\ K_2 = hf(x_i + \alpha h, y_i + \beta K_1) \end{cases}$$

将 $y(x_{i+1})$ 在 x_i 处泰勒展开有

$$y(x_{i+1}) = y(x_i + h) = y(x_i) + \dot{y}(x_i)h + \frac{1}{2!}\ddot{y}(x_i)h^2 + \frac{1}{3!}\dddot{y}(x_i)h^3 + O(h^4)$$

由 $\dot{y}(x) = f(x, y(x))$ 知

$$\begin{aligned} \ddot{y}(x) &= \dot{f}_x + \dot{f}_y \dot{y} = \dot{f}_x + \dot{f}_y f \\ \ddot{y}(x) &= \ddot{f}_{xx} + 2\ddot{f}_{xy}f + \ddot{f}_{yy}f^2 + \dot{f}_x \dot{f}_y + \dot{f}_y^2 f \end{aligned}$$

所以

$$\begin{aligned} y(x_{i+1}) &= y(x_i) + fh + \frac{1}{2}(\dot{f}_x + \dot{f}_y f)h^2 \\ &\quad + \frac{1}{6}(\ddot{f}_{xx} + 2\ddot{f}_{xy}f + \ddot{f}_{yy}f^2 + \dot{f}_x \dot{f}_y + \dot{f}_y^2 f)h^3 + O(h^4) \end{aligned}$$

其中 f 为 $f(x_i, y(x_i))$ 的简写，符号 $\dot{f}_x, \dot{f}_y, \ddot{f}_{xx}, \ddot{f}_{xy}, \ddot{f}_{yy}$ 的含义亦然。

又由二元函数的泰勒展开式有

$$\begin{aligned} K_2 &= hf(x_i + \alpha h, y_i + \beta K_1) \\ &= h \left(f + \alpha \dot{f}_x h + \beta \dot{f}_y f h + \frac{1}{2} \alpha^2 \ddot{f}_{xx} h^2 + \alpha \beta \ddot{f}_{xy} f h^2 + \frac{1}{2} \beta^2 \ddot{f}_{yy} f^2 h^2 + \cdots \right) \end{aligned}$$

于是

$$y_{i+1} = y_i + \lambda_1 K_1 + \lambda_2 K_2$$

$$\begin{aligned}
&= y_i + \lambda_1 f h + \lambda_2 h(f + \alpha \dot{f}_x h + \beta \dot{f}_y f h + \frac{1}{2} \alpha^2 \ddot{f}_{xx} h^2 \\
&\quad + \alpha \beta \ddot{f}_{xy} f h^2 + \frac{1}{2} \beta^2 \ddot{f}_{yy} f^2 h^2 + \dots) \\
&= y_i + (\lambda_1 + \lambda_2) f h + \lambda_2 (\alpha \dot{f}_x + \beta \dot{f}_y f) h^2 \\
&\quad + \lambda_2 \left(\frac{1}{2} \alpha^2 \ddot{f}_{xx} + \alpha \beta \ddot{f}_{xy} f + \frac{1}{2} \beta^2 \ddot{f}_{yy} f^2 \right) h^3 + O(h^4)
\end{aligned}$$

可得局部截断误差

$$\begin{aligned}
R[y] &= y(x_{i+1}) - y_{i+1} \\
&= (1 - \lambda_1 - \lambda_2) f h + \left[\left(\frac{1}{2} - \alpha \lambda_2 \right) \dot{f}_x + \left(\frac{1}{2} - \beta \lambda_2 \right) \dot{f}_y f \right] h^2 + \left[\left(\frac{1}{6} - \frac{1}{2} \alpha^2 \lambda_2 \right) \ddot{f}_{xx} \right. \\
&\quad \left. + \left(\frac{1}{3} - \alpha \beta \lambda_2 \right) \ddot{f}_{xy} f + \left(\frac{1}{6} - \frac{1}{2} \beta^2 \lambda_2 \right) \ddot{f}_{yy} f^2 + \frac{1}{6} (\dot{f}_x \dot{f}_y + \dot{f}_y^2 f) \right] h^3 + O(h^4)
\end{aligned}$$

为使 $R[y]$ 的阶尽可能高，由 f 的任意性，应选取 $\alpha, \beta, \lambda_1, \lambda_2$ ，使得 h, h^2 的系数为零，即得方程组

$$\begin{cases} 1 - \lambda_1 - \lambda_2 = 0 \\ \frac{1}{2} - \alpha \lambda_2 = 0 \\ \frac{1}{2} - \beta \lambda_2 = 0 \end{cases}$$

由此解出 $\alpha, \beta, \lambda_1, \lambda_2$ ，代入相应表达式即可。注意到 $\alpha, \beta, \lambda_1, \lambda_2$ 无论怎样选取， h^3 的系数中 $\dot{f}_x \dot{f}_y + \dot{f}_y^2 f \neq 0$ ，故 $R[y] = O(h^3)$ ，所以通过解上面的方程组得到的方法均是二阶方法。其含有3个方程，4个未知量，解有无穷个。

(1)取 $\lambda_1 = \lambda_2 = \frac{1}{2}, \alpha = \beta = 1$ ，则得

$$\begin{cases} y_{i+1} = y_i + \frac{1}{2} K_1 + \frac{1}{2} K_2 \\ K_1 = h f(x_i, y_i) \\ K_2 = h f(x_i + h, y_i + K_1) \end{cases}$$

即改进欧拉法。

(2)取 $\lambda_1 = 0, \lambda_2 = 1, \alpha = \beta = 1/2$ ，则可得

$$\begin{cases} y_{i+1} = y_i + K_2 \\ K_1 = h f(x_i, y_i) \\ K_2 = h f\left(x_i + \frac{1}{2} h, y_i + \frac{1}{2} K_1\right) \end{cases}$$

通常也写为

$$y_{i+1} = y_i + h f\left(x_i + \frac{1}{2} h, y_i + \frac{1}{2} h f(x_i, y_i)\right)$$

即变形欧拉法。

类似于上述二阶方法的推导，可得多种 4 级 4 阶 R-K 法。应用最广泛的是如下标准（经典）4 级 4 阶 R-K 法：

$$\begin{cases} y_{i+1} = y_i + \frac{1}{6}(K_1 + 2K_2 + 2K_3 + K_4) \\ K_1 = hf(x_i, y_i) \\ K_2 = hf\left(x_i + \frac{1}{2}h, y_i + \frac{1}{2}K_1\right) \\ K_3 = hf\left(x_i + \frac{1}{2}h, y_i + \frac{1}{2}K_2\right) \\ K_4 = hf(x_i + h, y_i + K_3) \end{cases}$$

另外两个常用的 4 级 4 阶 R-K 法的计算公式不再列出。

4 阶以上的 R-K 法计算函数值的工作量较大，对于大量的实际问题 4 阶 R-K 法已可满足精度要求。若有更高的精度要求，可使用 6 级 5 阶英格兰公式（此处不再给出）。

R-K 方法的优缺点如下：

优点：精度高，它是显式单步法，计算 y_{i+1} 只需要 y_i 一个值，每一步的步长可根据精度的要求单独考虑其大小。

缺点：需要计算多个函数值，当 $f(x, y)$ 比较复杂时，计算量较大。

八、程序框图

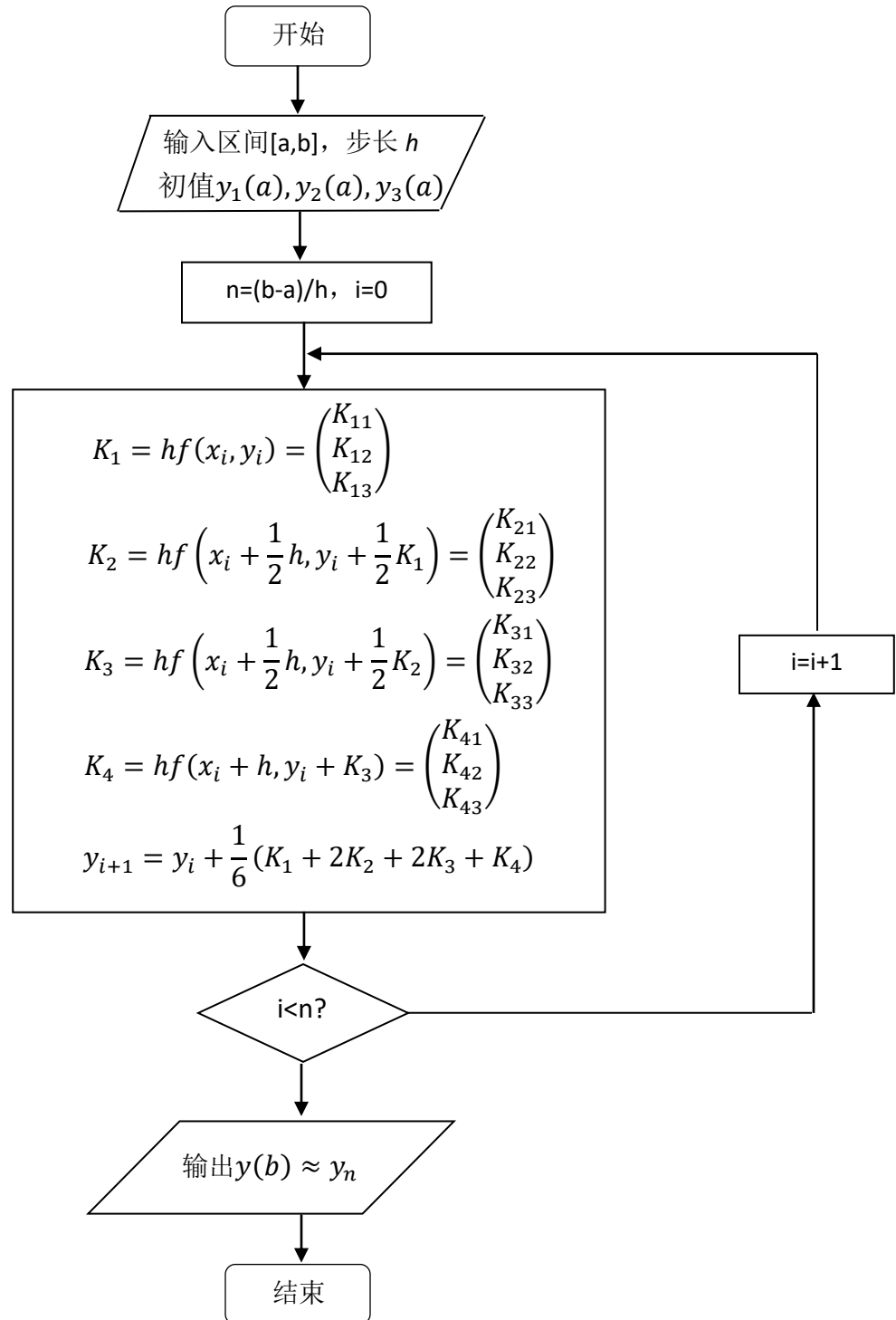
原题目是高阶微分方程初值问题，需将其转化为一阶微分方程组来求解。令 $y_1 = y, y_2 = \dot{y}, y_3 = \ddot{y}$, 转化后的方程如下：

$$\begin{cases} \dot{y}_1 = y_2 \\ \dot{y}_2 = y_3 \\ \dot{y}_3 = y_3 + y_2 - y_1 + 2x - 3 \\ y_1(0) = -1, y_2(0) = 3, y_3(0) = 3 \end{cases}$$

K_1, K_2, K_3, K_4 计算如下：

$$\begin{cases} K_1 = hf(x_i, y_i) = \begin{pmatrix} K_{11} \\ K_{12} \\ K_{13} \end{pmatrix} \\ K_2 = hf\left(x_i + \frac{1}{2}h, y_i + \frac{1}{2}K_1\right) = \begin{pmatrix} K_{21} \\ K_{22} \\ K_{23} \end{pmatrix} \\ K_3 = hf\left(x_i + \frac{1}{2}h, y_i + \frac{1}{2}K_2\right) = \begin{pmatrix} K_{31} \\ K_{32} \\ K_{33} \end{pmatrix} \\ K_4 = hf(x_i + h, y_i + K_3) = \begin{pmatrix} K_{41} \\ K_{42} \\ K_{43} \end{pmatrix} \end{cases}$$

程序框图如下：



三、程序使用说明及案例计算结果

该程序只针对案例 9.2 的微分方程，对其他微分方程需修改程序。该程序需要用户输入区间 $[a,b]$ 的左右端点，步长 h , y_1, y_2, y_3 的在 a 处的初值，然后可得案例 9.2 的 R-K 解和解析解，并计算了绝对误差和相对误差。应该注意输入的 a, b 之差应为步长 h 的整数倍。案例计算结果如下：



```
命令行窗口
请输入区间左端点: a=0
请输入区间左端点: b=1
请输入步长: h=0.05
请输入y1的初值: y1(a)=-1
请输入y2的初值: y2(a)=3
请输入y3的初值: y3(a)=2
R-K法解为y(b)=3.71828102
解析解为y(b)=1.00000000
解析解为y(b)=3.71828183
绝对误差为0.00000081
相对误差为0.00000022
fx >>
```

四、Matlab 程序（M-文件）

```
clear;clc;
a=input('请输入区间左端点:a=');
b=input('请输入区间左端点:b=');
h=input('请输入步长:h=');
y1=input('请输入 y1 的初值:y1(a)=');
y2=input('请输入 y2 的初值:y2(a)=');
y3=input('请输入 y3 的初值:y3(a)=');
y0=f(b);
y=RK4(a,b,y1,y2,y3,h);
d=y0-y;
m=d/y0;
fprintf('R-K 法解为 y(b)=%.8f\n',y);
fprintf('解析解为 y(b)=%.8f\n',b,y0);
fprintf('绝对误差为%.8f\n',d);
fprintf('相对误差为%.8f\n',m);

function[y0]=f(b)
y0=b*exp(b)+2*b-1;
```

end

function[y]=RK4(a,b,y1,y2,y3,h)

x=a;

n=(b-a)/h;

for i=0:n-1

 k11=h*y2;

 k12=h*y3;

 k13=h*(y3+y2-y1+2*(x+i*h)-3);

 k21=h*(y2+k12/2);

 k22=h*(y3+k13/2);

 k23=h*(y3+k13/2+y2+k12/2-(y1+k11/2)+2*((x+i*h)+h/2)-3);

 k31=h*(y2+k22/2);

 k32=h*(y3+k23/2);

 k33=h*(y3+k23/2+y2+k22/2-(y1+k21/2)+2*((x+i*h)+h/2)-3);

 k41=h*(y2+k32);

 k42=h*(y3+k33);

 k43=h*(y3+k33+y2+k32-(y1+k31)+2*((x+i*h)+h)-3);

 y1=y1+(k11+2*k21+2*k31+k41)/6;

 y2=y2+(k12+2*k22+2*k32+k42)/6;

 y3=y3+(k13+2*k23+2*k33+k43)/6;

end

y=y1;

end